

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Answers:.

1. Testing as an Engineering Activity

Software testing should be treated as a systematic engineering activity rather than simply executing random test cases. The test engineer should begin by understanding the requirements, identifying important test conditions, designing suitable test cases, defining expected results, executing the tests, and documenting the defects found. Test design techniques such as boundary value analysis and equivalence partitioning can be used to select effective test data. For example, instead of randomly testing a login system, the tester should systematically test valid credentials, invalid credentials, empty fields, incorrect passwords, and account-lock conditions. A structured approach provides better test coverage, reduces the possibility of missing important defects, and makes the testing process more consistent, measurable, and reliable.

2. Well-Defined Testing Process

A well-defined testing process provides a standard and systematic method for carrying out testing activities from planning to completion. It normally includes test planning, requirement analysis, test-case design, test execution, defect reporting, defect verification, and regression testing. In a banking system, testers can prepare specific test cases for login, balance checking, fund transfer, withdrawal, and transaction validation. When a defect is identified, it can be properly recorded and tracked until it is fixed and verified. A defined testing process reduces inconsistency, prevents important test activities from being skipped, improves defect detection, and increases the reliability and quality of the banking system.

3. Testing as a Structured SDLC Process

Testing should be performed throughout the Software Development Life Cycle rather than waiting until the software is completely developed. During the requirements phase, testers review requirements and identify ambiguities or missing conditions. During the design and development phases, test conditions and unit-level tests can be prepared. When individual modules are combined, integration testing verifies their interactions, followed by system testing of the complete application. After modifications or defect fixes, regression testing ensures that existing functionality continues to work correctly. Applying testing as a structured SDLC process helps identify defects earlier, reduces rework, and prevents integration and system-level defects from reaching customers.

4. Error, Defect and Failure

An error is a human mistake made during activities such as requirement analysis, design, coding, or testing. This mistake may introduce a defect or fault into the software. When the software containing the defect is executed and produces an incorrect or unexpected result, it causes a failure. For example, suppose a developer writes an incorrect condition while implementing login validation. The developer's mistake is the error, the incorrect condition in the program is the defect, and the system allowing an unauthorized user to log in is the failure. Therefore, an error can introduce a defect, and the execution of that defect can result in a failure.

5. Early Defect Detection

The testing principle of early testing should be applied to improve defect detection efficiency. Testing should begin as early as possible instead of waiting until the final stages of development. Testers can participate in requirement reviews, design reviews, and early test planning to identify problems before coding is completed. For example, if a payment requirement contains an incorrect calculation rule, identifying it during requirement review is much easier than discovering it after the complete payment system has been developed. Early defect detection reduces rework, saves development time and cost, and prevents defects from moving into later stages where they become more difficult to correct.

6. Tester Involvement in Requirement Phase

A tester should actively participate in the requirement analysis phase to improve the quality of the software from the beginning. The tester can review requirements and identify ambiguity, inconsistency, missing conditions, incomplete information, and requirements that cannot be properly tested. In a healthcare system, the tester can verify whether patient information, validation rules, data requirements, and expected outputs are clearly specified. Testers can also prepare initial test conditions based on the requirements. Early involvement helps prevent requirement-related defects from entering the design and coding stages, reduces rework, and improves communication between developers, testers, and other stakeholders.

7. Defects from Unclear Requirements

Defects can originate at the beginning of the software lifecycle when requirements are unclear, incomplete, ambiguous, or misunderstood. Developers may interpret the same requirement differently and implement functionality that does not match the actual business need. For example, if a payroll system does not clearly specify how overtime payment should be calculated, the developer may implement an incorrect calculation and produce an incorrect salary. This defect may then continue through design, coding, and testing. Clear, complete, consistent, and testable requirements help developers understand the expected behavior correctly and allow testers to create accurate test cases, preventing many defects before implementation.

8. Boundary Value Analysis

A system that crashes when users enter boundary values indicates that boundary conditions may not have been adequately tested. Boundary Value Analysis (BVA) is a testing technique that focuses on values at and around the limits of an input range because defects frequently occur near boundaries. For example, if an application accepts an age between 18 and 60, the tester should test values such as 17, 18, 19, 59, 60, and 61. Testing only normal values such as 30 may not reveal boundary-related defects. Therefore, proper boundary testing could have detected the crash earlier and improved the test coverage of the system.

9. Defect Repository

A defect repository is a centralized collection of information about previously identified software defects. It can contain details such as the defect description, affected module, severity, priority, cause, steps to reproduce, and resolution. Testers can analyze this historical information while designing new test cases. For example, if previous releases repeatedly contained validation defects in a particular module, additional

validation test cases can be created for that module in future releases. Previous defect scenarios can also be included in regression testing. Thus, a defect repository supports better test-case design, prevents repeated defects, and improves the overall quality of the software.

10. Functional Defect in ATM

An ATM withdrawal failure caused by incorrect transaction logic is classified as a functional defect because the system is not performing the required withdrawal function according to its specification. The tester should design test cases for sufficient balance, insufficient balance, valid withdrawal amounts, invalid amounts, daily limits, balance updates, and transaction cancellation. Integration testing should also verify the communication between the ATM interface, account system, and transaction processing system. If these scenarios had been tested properly, the incorrect transaction logic could have been detected before release. Regression testing after fixing the defect would further ensure that the problem does not occur again.

11. Defect Clustering

Defect clustering is the concept that a small number of modules or components may contain a large proportion of the defects in a software system. The testing team should analyze historical defect information and identify modules where defects are highly concentrated. For example, if most defects are found in the payment module, that module should receive additional test cases, reviews, and testing effort. High-risk modules can be prioritized instead of distributing testing effort equally across every module. This helps the team use limited testing resources effectively, increases the possibility of finding important defects, and improves overall testing efficiency.

12. Performance Defect

A web application that becomes slow or unresponsive when many users access it simultaneously has a performance defect. This defect is related to the system's response time, capacity, or behavior under load rather than simply whether a function works correctly. Testers can perform load testing, stress testing, and performance testing to identify the problem. They can measure response time, throughput, and system behavior under different numbers of users. Proper defect classification helps the development team understand the nature of the problem and select an appropriate solution, such as improving database queries or optimizing system resources.

13. Historical Defect Data for Regression Testing

Historical defect data can be effectively reused when preparing regression tests for new software releases. Testers can examine previously reported defects and identify the conditions that caused them. These conditions can then be converted into permanent regression test cases and executed whenever related modules are modified. For example, if a payment module previously failed during a particular transaction condition, that scenario should be included in future regression testing. Reusing historical defect information improves test coverage, helps prevent previously fixed defects from returning, and reduces the time required to design completely new test cases.

14. Tester Role in Integration

A tester plays an important role in minimizing integration defects by checking whether different modules communicate and exchange information correctly. The tester should understand module interfaces, identify possible data-flow problems, prepare integration test cases, and verify the expected behavior when modules interact. Early communication between developers and testers helps clarify requirements and interface expectations before integration begins. For example, if one module sends data in a format that another module cannot process, early testing can identify the mismatch. Therefore, early tester involvement reduces communication gaps and helps prevent integration defects.

15. Defect Prevention

Defect prevention focuses on identifying and removing the root causes of defects instead of repeatedly fixing the same problems after they occur. The team can analyze recurring defects, perform root-cause analysis, conduct requirement and design reviews, improve coding practices, and strengthen testing procedures. For example, if every release contains similar input-validation defects, the team should determine why validation requirements or test cases are being missed and improve the process accordingly. Defect prevention reduces recurring problems, lowers maintenance effort, and improves software quality over the long term.

16. Structured Testing for Mobile Applications

A mobile application should follow a standardized testing strategy rather than using different or inconsistent testing approaches for each release. The testing process should include functional testing, compatibility testing, usability testing, performance testing, and crash testing. Test cases should be properly documented, executed consistently, and linked with requirements where appropriate. Testing should also cover different devices, screen sizes, operating system versions, and network conditions. Regression testing should be performed after changes to ensure existing features continue to work. A structured approach improves consistency, increases coverage, reduces missed defects, and improves application reliability.

17. Testing and Debugging

Testing and debugging are related activities, but they have different purposes. Testing is performed to identify whether the software contains defects by comparing actual results with expected results. Debugging begins after a defect is identified and involves locating its root cause and correcting the program. For example, if an online banking application displays an incorrect transaction amount, the tester reports the failure during testing. The developer then examines the relevant code, identifies the incorrect calculation, and fixes it through debugging. Therefore, testing mainly detects defects, while debugging finds and removes their causes.

18. Cross-Browser Testing

Cross-browser testing is performed to verify that a web application behaves consistently across different browsers and browser versions. The tester should execute important functional, interface, navigation, and compatibility test cases using commonly supported browsers such as Chrome, Edge, and Firefox. The tester

should check whether buttons, forms, layouts, scripts, and other features behave correctly in each browser. If the application works correctly in one browser but fails in another, the compatibility issue should be recorded and corrected. Cross-browser testing reduces customer dissatisfaction and ensures a consistent user experience.

19. Validation in Logistics Application

Improper validation can allow incorrect or unexpected information to enter a logistics application and produce inconsistent outputs. Testers should identify validation gaps by testing valid inputs, invalid inputs, empty fields, incorrect formats, boundary values, and unexpected data. For each test, the actual result should be compared with the expected result. For example, if a delivery quantity field accepts only positive values, zero, negative numbers, and invalid characters should be tested. Proper validation prevents incorrect data from being processed and improves the reliability, consistency, and correctness of the application.

20. Systematic Test Design

A team should not directly execute random tests because random testing can result in poor coverage and may miss important conditions. Testing should be treated as an engineering activity in which requirements are analyzed, test conditions are identified, test cases are designed, expected results are defined, tests are executed, and defects are documented. For an e-commerce application, planned test cases should cover login, product search, product selection, cart operations, payment, and order confirmation. Systematic test design provides better coverage, makes testing repeatable, and increases the effectiveness of defect detection.

21. Integration Issues and Data Consistency

Data inconsistency between healthcare modules can occur because of incorrect interfaces, incompatible data formats, communication problems, or errors in integration logic. Testers should verify whether the data sent from one module is correctly received, processed, and displayed by another module. Integration test cases should cover normal data, missing data, invalid data, and different communication conditions. For example, if a patient's updated information is saved in one module but an older value is displayed in another module, integration testing should identify the problem. Proper integration testing ensures consistent and accurate data across the system.

22. Tester Role Across SDLC

Testers should be involved throughout the SDLC rather than only during the final testing stage. During requirement analysis, they review requirements and identify ambiguities; during design, they identify test conditions; during development, they prepare and perform appropriate tests; during integration, they verify interactions between modules; and during system testing, they validate the complete application. After changes or defect fixes, regression testing is performed. Early tester involvement improves communication, helps identify defects sooner, reduces rework, and prevents project delays caused by late discovery of problems.

23. Test Coverage

Test coverage refers to how much of the software functionality, requirements, and important scenarios are covered by testing. A system that fails during peak usage may indicate that important load, stress, boundary, or exceptional scenarios were not included in the test cases. Testers should identify critical business functions and create test cases for normal usage, peak usage, invalid inputs, boundary conditions, and unexpected situations. Better test coverage increases the possibility of finding critical defects before production and reduces the risk of failures when the system is used by real customers.

24. Defect Repository and Documentation

A proper defect repository should maintain complete and organized information about every reported defect. Important information includes defect ID, description, affected module, severity, priority, steps to reproduce, expected result, actual result, assigned person, status, and resolution. This documentation allows testers and developers to track each defect from identification through closure. It also helps management analyze defect trends and identify modules with repeated problems. By maintaining accurate defect records, the organization can improve test planning, tracking, root-cause analysis, and future software quality.

25. Structured Testing for Banking Systems

Banking systems require a highly structured testing approach because failures in financial transactions can seriously affect users and organizations. Testers should systematically test login, account balance, deposits, withdrawals, fund transfers, transaction limits, error handling, and transaction status. Functional testing verifies individual functions, integration testing checks communication between banking modules, system testing validates the complete system, and regression testing ensures that changes do not break existing functions. A structured testing process improves reliability, increases defect detection, and reduces repeated transaction failures.

26. Edge Case and Boundary Testing

Testing only normal inputs is not sufficient because defects often occur at the edges of acceptable input ranges. Testers should include minimum values, maximum values, values just below and above the limits, and other important edge cases. For example, if a system accepts values from 1 to 100, suitable test values include 0, 1, 2, 99, 100, and 101. These tests can reveal unexpected failures that normal inputs may not detect. Boundary and edge-case testing therefore improves test coverage and helps identify defects before the system reaches users.

27. Process-Based Testing

A process-based testing approach ensures that testing activities are performed according to standardized procedures across different modules and teams. The organization can define common methods for test planning, test-case design, execution, documentation, defect reporting, and regression testing. When all testers follow the same process, the quality of testing becomes more consistent and easier to monitor. For example, every module can use the same defect-reporting format and test-case structure. Standardization

reduces uneven testing quality, prevents important activities from being skipped, and produces more reliable testing outcomes.

28. Input Validation

A web application should properly validate all user inputs before processing them. Testers should verify normal, invalid, empty, boundary, unexpected, and incorrectly formatted values. For example, if a field accepts only numerical values, alphabetic characters and inappropriate symbols should be rejected or handled correctly. Testers should also verify that invalid inputs do not cause crashes or unexpected application behavior. Proper input validation ensures that only acceptable data is processed, improves system robustness, and helps prevent failures caused by unexpected user input.

29. Defect Leakage

Defect leakage occurs when a defect is not detected during the testing stages and reaches the production environment. Common causes include incomplete requirements, inadequate test coverage, poorly designed test cases, insufficient regression testing, and late involvement of testers. To reduce defect leakage, the team should perform early requirement reviews, use systematic test design, improve test coverage, analyze previous defects, and conduct regression testing after changes. Regular review of escaped defects can also help identify weaknesses in the testing process. These practices reduce production defects and improve customer confidence.

30. Early Defect Detection

A testing strategy should focus on identifying defects as early as possible through requirement reviews, design reviews, unit testing, integration testing, and system testing. Early testing prevents defects from moving into later stages where they may become more difficult and expensive to correct. For example, identifying an incorrect requirement during a review requires only a requirement change, whereas discovering the same problem after deployment may require code modification, retesting, redeployment, and customer support. Therefore, early defect detection reduces rework, saves time and cost, improves efficiency, and contributes to better software quality.



Code of Conduct:

I Swathi S 192421111 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand